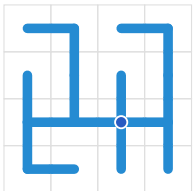
Pipes & Light Up

Search algorithms for two logic puzzles

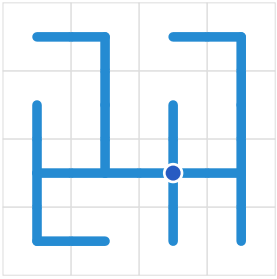


Introduction to Artificial Intelligence — Assignment 1
(HK1 2026–2027)

DFS (blind) + Greedy Best-First (Pipes) · DFS + MRV/CP (Light Up)

[Group — member names go here]

Goal: one connected, acyclic network



- Every port matches its neighbor.
- No port leaves the grid.
- All tiles connect. No cycles.

Tiles are vertices, connections are edges.

$$G \text{ is a tree} \implies |E| = n - 1$$

The blue source anchors water visualization.

Rules: puzzle-pipes.com. Goal validation: model.py and search.py.

Problem formulation

Input	$n = m^2$ tiles, fixed shapes, ordinary boundaries
State	Prefix $s = (a_0, \ldots, a_{k-1})$ of assigned orientations
Initial state	Empty prefix (), with all tile shapes known
Action	Assign a rotation to tile k in row-major order
Transition	$s' = s + (a_k)$, accepted after feasibility checks
Goal	$k = n$ and the complete board passes all rules
Cost	Unit assignment cost, unused by either ranking

A solver assignment can represent several manual clicks.
Any valid solution is acceptable. Minimum-click solutions are outside scope.

Implemented state model: Search in pipes_problem/search.py.

Legal successors

The next tile has a fixed shape. Search chooses its orientation.



NE
reject



ES
keep



SW
reject



NW
reject

Top-left corner example. The left and top borders forbid N and W.

Neighbor matching also checks absent ports.
A candidate must pass every shared pruning check before search accepts it.

Example is schematic. Orientation filtering: boundary_domains() and compatible().

Search space

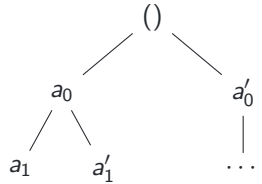
Distinct rotations per tile

End, elbow, T-junction	4
Straight	2
Cross	1

$$U = \prod_{i=0}^{n-1} |R_i| \leq 4^n$$

Complete assignments before boundary checks.

Prefix tree

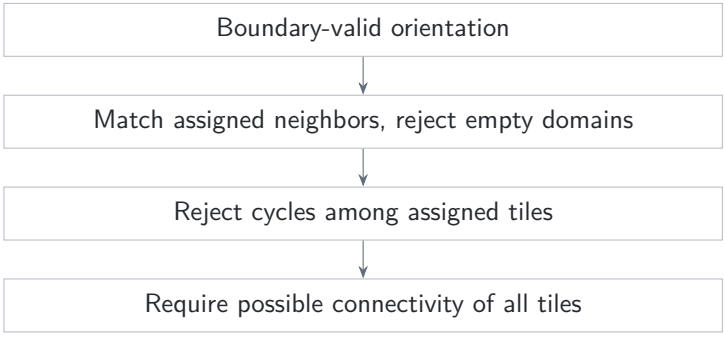


Depth n, branching factor at most 4.
Each prefix has one parent.

For the shipped 4×4 example: $U = 268{,}435{,}456$.
This bound counts assignments, not measured expansions.

Count derived from distinct rotations in pipes-4x4-261.json.

Shared pruning



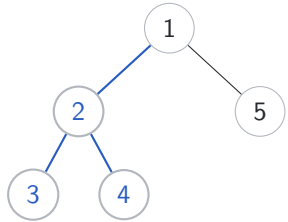
Both solvers use identical checks and fixed tile order.

Root condition: total ports $= 2(n-1)$.
Possible connectivity is necessary but does not prove a solution exists.

Pruning implementation: Search.compatible(). Full reasoning: docs/pipes-analysis.md.

Depth-first search

```
VISIT(s):
  if budget exhausted: return LIMIT
  count expansion
  if GOAL(s): return SOLVED(s)
  for a in rotations of tile |s|:
    child = s + (a)
    if FEASIBLE(child):
      r = VISIT(child)
      if r is SOLVED or LIMIT: return r
  return UNSOLVABLE
```



Schematic visit order.
Explore a branch, then backtrack.

Finite-tree completeness with sufficient resources and sound pruning.
Worst-case exploration remains exponential.

Simplified pseudocode matches dfs(). Root feasibility precedes VISIT. Cancellation is external.

Greedy's heuristic: remaining choices

$$h(s) = \sum_{i \text{ unassigned}} \log_2 |D_i(s)|$$

$D_i(s)$ contains boundary-valid orientations matching assigned neighbors.

Candidate	Remaining domain sizes	$h(s)$
A	$(1, 2, 4)$	$0 + 1 + 2 = 3$
B	$(1, 1, 2)$	$0 + 0 + 1 = 1$

Greedy prefers B. Smaller values indicate fewer unresolved choices.
Tie order: deeper prefix, then earlier insertion.

The score ignores dependencies between unassigned tiles.
It provides no admissibility or minimum-click guarantee.

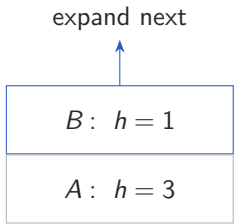
Hypothetical domain example. Exact priority: (h, -depth, insertion order) in greedy().

Greedy Best-First search

```
push () into min-priority queue
while queue is not empty:
  if budget exhausted: return LIMIT
  s = pop minimum-priority state
  count expansion
  if GOAL(s): return SOLVED(s)
  for a in rotations of tile |s|:
    child = s + (a)
    D = FEASIBLE(child)
    if D exists:
      push child with (h(child),
                       -|child|, serial)
return UNSOLVABLE
```

Root feasibility precedes the queue. Priority uses h alone. The unique root starts at zero.

Queue selection



All feasible prefixes remain queued.

Memory can grow exponentially.

Evaluation protocol

- Same nine saved maps for both algorithms.
- Three maps per size, five repeats per algorithm.
- Shared pruning, tile order and resource limits.

Saved map → Timed headless solve → Separate memory replay

Time excludes drawing and playback. Peak memory counts traced Python allocations. Combine passes only when outcomes agree.

Limits: 30 seconds of solver computation and 200,000 expansions.

Runner: benchmark.py. Measurement: measure() and worker.py.

Baseline results

Size	Algorithm	Solved/runs	Time (ms)	Peak (KiB)
4×4	DFS	15/15	3.149	26.984
	Greedy	15/15	3.097	12.359
5×5	DFS	15/15	5.261	53.297
	Greedy	15/15	6.157	19.219
7×7	DFS	15/15	24.469	145.852
	Greedy	15/15	31.695	27.594

Median over 15 successful runs per size and algorithm.
Each group contains three distinct inputs with five timing repeats.

DFS wins at 5×5 and two of three 7×7 maps; Greedy takes the hardest map (38.2 vs 40.8 ms) and holds 4–5× less traced memory.
These inputs are easy under shared pruning; no general winner is claimed.

Measured 25 October 2026, Linux, Python 3.13. Raw rows: docs/benchmarks/2026-10-25-pipes.csv.

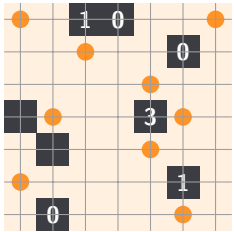
Scope and next experiments

- Compare harder maps and proven-unsolvable inputs.
- Measure the effect of shared pruning separately.
- Keep UNSOLVABLE distinct from LIMIT or CANCELLED.
- Try alternative heuristics (MRV) and broader input sets.

References

Pipes rules: `https://www.puzzle-pipes.com/`
Russell and Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Chapters 3 and 6.
`https://aima.cs.berkeley.edu/`

Problem 2: Light Up (Akari)



- Light every white cell; bulbs shine until a wall or edge.
- No two bulbs may see each other.
- Each numbered wall has exactly its count of adjacent bulbs.
- Compare **DFS** with **Backtracking + MRV + CP** for a first solution.

Current 7×7 easy sample, seed 70097; validated CSP solution.

Generator: walls-first-witness-v2.

Formulation and the comparison

- State: W-bit bulb mask, lit mask, legal-location mask.
- Initial state: no bulbs. Goal: full lighting, exact clues, no conflicts.
- DFS: first dark cell; row-major candidate ordering; descend immediately.
- CSP: fixed-point propagation, dynamic MRV, lighting-gain value ordering.
- Same boards and budgets; different pruning and per-node work.

Neither DFS nor CSP guarantees minimum bulb count.
The experiment measures the combined MRV/CP solver, not an isolated MRV ablation.

Implementation: light_up_problem/algorithms/search.py.

DFS: accept the first feasible bulb and descend

```
DFS(state):
  check budgets
  if GOAL(state): return SOLVED
  target = first dark cell, row-major
  for bulb in legal illuminators(target):
    if FEASIBLE(place(bulb)):
      search child immediately
      if solved or limited: return result
    exclude bulb from later siblings
  return UNSOLVABLE
```

Explicit frames retain untried candidates; parent links reconstruct the path. Backtrack only after the current subtree fails. Worst-case search is exponential.

depth_first_search(); TIMEOUT / NODE_LIMIT / MEMORY_LIMIT are not UNSOLVABLE.

MRV chooses a constraint; CP forces deductions

- Wall with u available neighbors and deficit r: $\binom{u}{r}$ choices.
- Dark cell: one choice per legal illuminator. Choose the smallest domain; prefer walls on ties; try greater lighting gain first.
- CP: satisfied wall $\Rightarrow$ forbid remaining neighbors.
- Deficit equals available neighbors $\Rightarrow$ force all their bulbs.
- One legal illuminator $\Rightarrow$ force that bulb; exclude its sight lines.
- Diagonal deductions remove choices that starve a numbered wall.

Repeat until no change; any contradiction rejects the branch.

_View.select_mrv_choices() and _View.propagate().

Backtracking + MRV + Constraint Propagation

```
CSP(state):
  check budgets; propagate to fixed point
  if contradiction: reject branch
  if GOAL(state): return SOLVED
  constraint = minimum remaining choices
  for choice ordered by lighting gain:
    apply choice; propagate
    if consistent: descend immediately
    if solved or limited: return result
    restore parent checkpoint
  return UNSOLVABLE
```

A checkpoint restores both bulbs and legal domains. A CSP node can contain multiple bulb placements, so node counts alone do not measure equal work.

backtracking_mrv_cp(); explicit stack avoids Python recursion limits.

Inputs: five different seeds per configuration

- Walls first; place witness bulbs on randomly chosen dark cells until lit.
- Derive clues; keep about 25% of walls unnumbered; independently validate witness.
- Sizes 7/10/14/25 $\times$ easy/normal/hard $\times$ 5 seeds = **60 boards**.
- Wall density: 16% / 10.5% / 5.5%; uniqueness is not checked.
- Both solvers receive the same exact board for every seed.

Seed = size $\times$ 10,000 + difficulty index $\times$ 97 + offset (0–4).
Difficulty indices are 1/2/3. These are five inputs, not five repeats of one input.

Exact JSON, seeds and SHA-256 hashes: 2026-10-31-lightup-dfs-csp-manifest.json.

Visual demonstration

- Select DFS or CSP (MRV + CP), Start Search, Pause, Resume, arrows.
- Full event history at **2 / 8 / 30 steps/s**; no dropped steps.
- CSP: each forced bulb and domain reduction, including initial propagation.
- Forbidden dark cells show $\times$ independently of manual notes.
- Backtracking restores bulbs and domains; final result waits for the trace.

Spawned workers keep the UI responsive. Visualization has no timeout; node/memory caps remain. The experiment retains all three limits.

ui/app.py + worker.py. The Compare button compares DFS and CSP.

Measurement protocol: 120 runs, unchanged limits

- 60 boards $\times$ DFS/CSP; one run per algorithm per seed; sequential fresh processes.
- Alternate algorithm order on successive seeds. No animation or trace.
- **30 s / 500,000 expanded nodes / 512 MiB estimated memory** for both.
- Solver time with tracemalloc active; peak Python allocation, not RSS.
- Generation/startup excluded; independently validate every SOLVED result.
- Record `TIMEOUT`, `NODE_LIMIT`, `MEMORY_LIMIT` separately.

A stopped run is neither a completed solve nor proof of unsolvability.
Table times are medians of all attempts; solved-only medians are in the JSON summary.

31 October 2026; CSV + summary + environment/source/input manifest under docs/benchmarks/.

Results: 7 / 10

	DFS			CSP (MRV + CP)		
Size / level	S/5	s	KiB	S/5	s	KiB
7 easy	5/5	0.0075	24.9	5/5	0.0066	25.2
7 normal	5/5	0.0066	26.8	5/5	0.0103	29.0
7 hard	5/5	0.0074	27.3	5/5	0.0107	30.5
10 easy	5/5	0.0229	55.9	5/5	0.0288	56.2
10 normal	5/5	0.0218	60.0	5/5	0.0309	61.8
10 hard	5/5	0.0229	62.4	5/5	0.0314	67.8

S: SOLVED runs. Medians over 5 seeds, including duration of stopped attempts.
KiB is traced peak allocation, not RSS or the solver memory estimate.

31/10/2026; 2026-10-31-lightup-dfs-csp.csv; 30 s / 500,000 nodes / 512 MiB.

Results: 14 / 25

		DFS			CSP (MRV + CP)	
Size / level	S/5	s	KiB	S/5	s	KiB
14 easy	5/5	0.2330	135.3	5/5	0.0801	109.1
14 normal	5/5	0.0724	117.3	5/5	0.1030	123.5
14 hard	5/5	0.0832	126.1	5/5	0.1168	134.6
25 easy	2/5	30.0001	2237.0	5/5	0.9558	562.1
25 normal	1/5	30.0004	3583.3	5/5	1.0274	696.9
25 hard	4/5	0.7835	870.3	5/5	1.0421	868.0

S: SOLVED runs. Medians over 5 seeds, including duration of stopped attempts.
KiB is traced peak allocation, not RSS or the solver memory estimate.

31/10/2026; 2026-10-31-lightup-dfs-csp.csv; 30 s / 500,000 nodes / 512 MiB.

Interpreting the measured results

- Solved: **DFS 52/60**, **CSP 60/60**; TIMEOUT: 8 / 0.
- NODE_LIMIT / MEMORY_LIMIT: **0 each** for both solvers; both caps remain enabled.
- Among 52 jointly solved pairs, CSP is faster on 15; median paired $t_{DFS}/t_{CSP} = 0.73$. Excludes stopped pairs.
- Small well-pruned boards may favor DFS; CP can cost more per node.
- CSP can reduce branching substantially. A node can include many forced bulbs.

Five seeds are a small sample; wall density does not order solve difficulty.

New DFS/CSP data use the current generator; older measurements use an earlier generator.

Limitations and next experiments

- No optimal-bulb or uniqueness claim for these first-solution solvers.
- Tracemalloc adds overhead and does not measure RSS.
- Five seeds per configuration: expand the sample and add unsolvable boards.
- Ablate CP/MRV and measure uninstrumented runtime separately.

Rules: `https://www.puzzle-light-up.com/`
Russell & Norvig, *Artificial Intelligence: A Modern Approach*, Ch. 3 and 6.
Implementation and analysis: `docs/lightup-analysis.md`.

Summary

- Pipes: DFS / Greedy. Updated Light Up experiment: DFS / MRV + CP.
- Visual demos show complete search steps; CSP also shows forced deductions.
- New Light Up evidence: 60 distinct boards, 120 paired runs, unchanged limits.
- Report solve rates and TIMEOUT / NODE_LIMIT / MEMORY_LIMIT separately.
- Data: `docs/benchmarks/2026-10-31-lightup-dfs-csp.csv`.

References

`https://www.puzzle-pipes.com/` · `https://www.puzzle-light-up.com/`
Russell & Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed.
Implementation and analysis: `pipes_problem/`, `light_up_problem/`, `docs/`.